

Capítulo

3

PILAS Y COLAS

3.1 INTRODUCCIÓN

Cuando se presentaron los arreglos, en el capítulo 1, se mencionó que eran estructuras lineales. Es decir, cada componente tiene un único sucesor y un único predecesor con excepción del primero y del último, respectivamente. Por otra parte, al analizar las operaciones de inserción y eliminación, se observó que los elementos se podían insertar o eliminar en cualquier posición del arreglo. Cabe señalar, sin embargo, que existen problemas que por su naturaleza requieren que los elementos se agreguen o se quiten sólo por un extremo. Este capítulo se dedica al estudio de **pilas** y **colas**, que son estructuras de datos lineales con restricciones en cuanto a la posición en la cual se pueden llevar a cabo las operaciones de inserción y eliminación de componentes.

3.2 PILAS

Una **pila** representa una estructura lineal de datos en la que se puede agregar o quitar elementos únicamente por uno de los dos extremos. En consecuencia, los elementos de una pila se eliminan en orden inverso al que se insertaron; es decir, el último elemento que se mete en la pila es el primero que se saca. Debido a esta característica, se le conoce como estructura **LIFO** (*Last-Input, First-Output*: el último en entrar es el primero en salir).

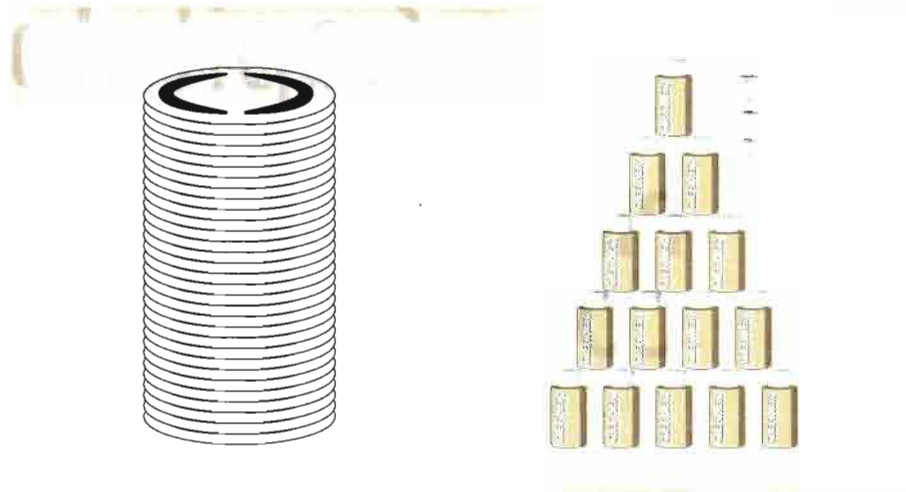
Existen numerosos casos prácticos en los que se utiliza el concepto de pila; por ejemplo, una pila de platos, una pila de latas en un supermercado, una pila de libros que se exhiben en una librería, etcétera. En la figura 3.1 se observa una pila de platos. Es de suponer que si el cocinero necesita un plato limpio, tomará el que está encima de todos, que es el último que se colocó en la pila.

Las **pilas** son estructuras de datos lineales, como los arreglos, ya que los componentes ocupan lugares sucesivos en la estructura y cada uno de ellos tiene un único sucesor y un único predecesor, con excepción del último y del primero, respectivamente.

Una **pila** se define formalmente como una colección de datos a los cuales se puede acceder mediante un extremo, que se conoce generalmente como tope.

FIGURA 3.1

Ejemplos prácticos de pilas.



3.2.1 Representación de pilas

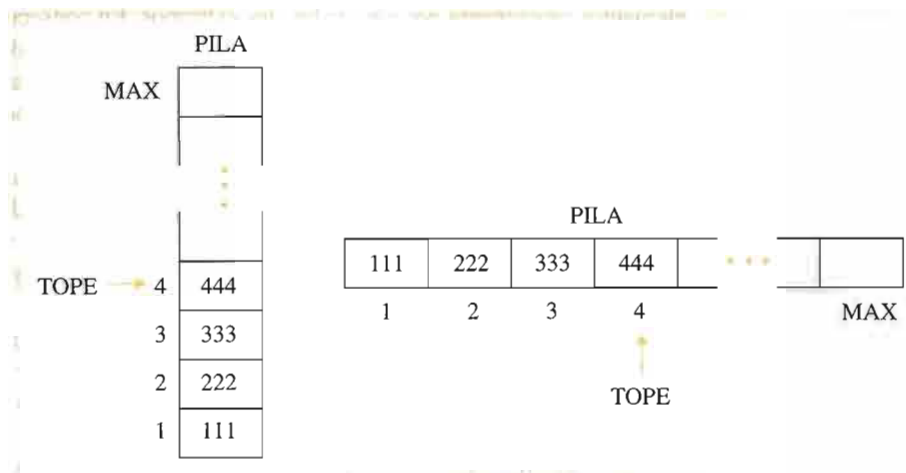
Las pilas no son estructuras fundamentales de datos; es decir, no están definidas como tales en los lenguajes de programación. Para su representación requieren el uso de otras estructuras de datos, como:

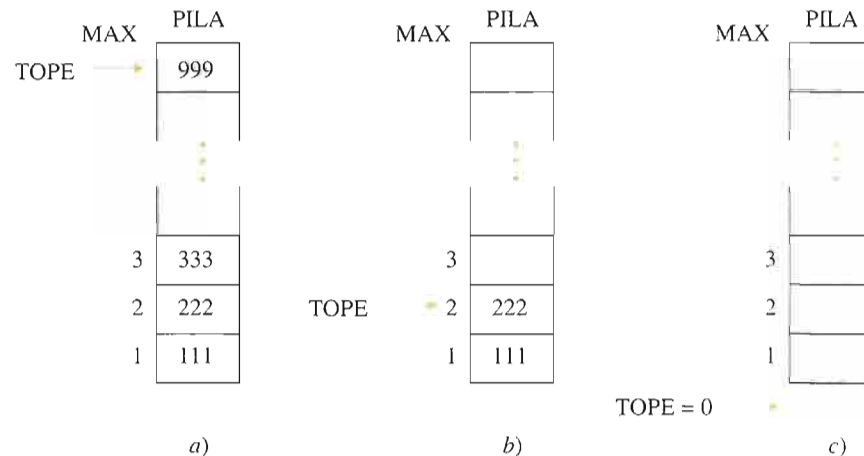
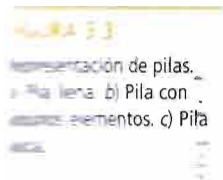
- ▮ Arreglos
- ▮ Listas

En este libro se utilizarán arreglos. En consecuencia, es importante definir el tamaño máximo de la pila, así como una variable auxiliar a la que se denomina TOPE. Esta variable se utiliza para indicar el último elemento que se insertó en la pila. En la figura 3.2 se presentan dos alternativas de representación de una pila, utilizando arreglos.

FIGURA 3.2

Representación de pilas.





En la figura 3.3 se presentan ejemplos de *a)* pila llena, *b)* pila con algunos elementos y *c)* pila vacía.

Al utilizar arreglos para implementar pilas se tiene la limitación de que se debe reservar espacio de memoria con anticipación, característica propia de los arreglos. Una vez dado un máximo de capacidad a la pila no es posible insertar un número de elementos mayor al máximo establecido. Si la pila estuviera llena y se intentara insertar un nuevo elemento, se producirá un error conocido como **desbordamiento** —*overflow*—. Por ejemplo, si en la pila que se presenta en la figura 3.3*a*, donde $\text{TOPE} = \text{MAX}$, se quisiera insertar un nuevo elemento, se producirá un error de este tipo. La pila está llena y el espacio de memoria reservado es fijo, no se puede expandir ni contraer.

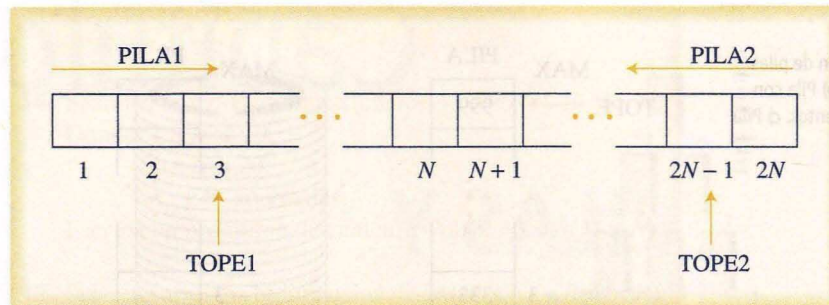
Una posible solución a este tipo de inconvenientes consiste en definir pilas de gran tamaño, pero esto último resultaría ineficiente y costoso si sólo se utilizaran algunos elementos. No siempre es viable saber con exactitud cuál es el número de elementos a tratar; por tanto, siempre existe la posibilidad de cometer un error de desbordamiento —si se reserva menos espacio del que efectivamente se usará— o bien de hacer uso ineficiente de la memoria —si se reserva más espacio del que realmente se necesita—.

Existe otra alternativa de solución a este problema. Consiste en usar **espacios compartidos** de memoria para la implementación de pilas. Supongamos que se necesitan dos pilas, cada una de ellas con un tamaño máximo de N elementos. Se definirá entonces un solo arreglo unidimensional de $2 * N$ elementos, en lugar de dos arreglos de N elementos cada uno.

Como se ilustra en la figura 3.4, la PILA1 ocupará desde la posición 1 en adelante (2, 3, ...), mientras que la PILA2 ocupará desde la posición $2*N$ hacia atrás ($2*N - 1$, $2*N - 2$, ...). Si en algún punto del proceso la PILA1 necesitara más espacio del que realmente tiene — N — y en ese momento la PILA2 no tuviera ocupados sus N lugares, entonces sería posible agregar elementos a la PILA1 sin caer en un error de desbordamiento (figura 3.5). Algo similar podría suceder para la PILA2, si ésta necesitara más de N espacios y la PILA1 tuviera lugares disponibles (figura 3.5*b*).

FIGURA 3.4

Representación de pilas en espacios compartidos.



Otro error que se puede presentar al trabajar con pilas es tratar de eliminar un elemento de una pila vacía. Este tipo de error se conoce como **subdesbordamiento** —*underflow*—. Por ejemplo, si en la pila que se presenta en la figura 3.3c, donde $\text{TOPE} < 1$, se deseara eliminar un elemento, se presentaría un error de este tipo.

3.2.2 Operaciones con pilas

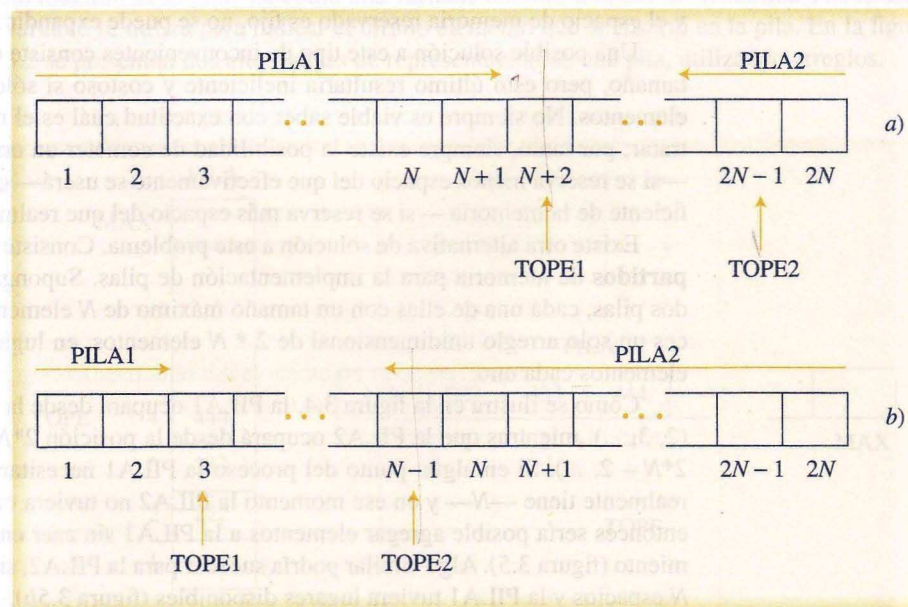
La definición de una estructura de datos queda completa al incluir las operaciones que se pueden realizar en ella. Para el caso de las pilas, las operaciones básicas que se pueden llevar a cabo son:

- Insertar un elemento —*Push*— en la pila
- Eliminar un elemento —*Pop*— de la pila

FIGURA 3.5

Representación de pilas en espacios compartidos.

- a) PILA1 tiene más de N elementos y PILA2 tiene menos de N elementos.
 b) PILA2 tiene más de N elementos y PILA1 tiene menos de N elementos.



Y las operaciones auxiliares:

- ▶ Pila_vacía
- ▶ Pila_llena

Considerando que se tiene una pila con capacidad para almacenar un número máximo de elementos —MAX—, y que el último de ellos se indica con TOPE, a continuación se presentan los algoritmos correspondientes a las operaciones mencionadas. Si la pila está vacía, entonces TOPE es igual a 0.

Algoritmo 3.1 Pila_vacía

Pila_vacía (PILA, TOPE, BAND)

{Este algoritmo verifica si una estructura tipo pila —PILA— está vacía, asignando a BAND el valor de verdad correspondiente. La pila se implementa en un arreglo unidimensional. TOPE es un parámetro de tipo entero. BAND es un parámetro de tipo booleano}

1. Si (TOPE = 0) {Verifica si no hay elementos almacenados en la pila}
 entonces
 Hacer BAND ← VERDADERO {La pila está vacía}
 si no
 Hacer BAND ← FALSO {La pila no está vacía}
2. {Fin del condicional del paso 1}

Algoritmo 3.2 Pila_llena

Pila_llena (PILA, TOPE, MAX, BAND)

{Este algoritmo verifica si una estructura tipo pila —PILA— está llena, asignando a BAND el valor de verdad correspondiente. La pila se implementa en un arreglo unidimensional de MAX elementos. TOPE es un parámetro de tipo entero. BAND es un parámetro de tipo booleano}

1. Si (TOPE = MAX)
 entonces
 Hacer BAND ← VERDADERO {La pila está llena}
 si no
 Hacer BAND ← FALSO {La pila no está llena}
2. {Fin del condicional del paso 1}

Algoritmo 3.3 Pone

Pone (PILA, TOPE, MAX, DATO)

{Este algoritmo agrega el elemento DATO en una estructura tipo pila —PILA—, si la misma no está llena. Actualiza el valor de TOPE. MAX representa el número máximo de elementos que puede almacenar PILA. TOPE es un parámetro de tipo entero}

1. Llamar a Pila_llena con PILA, TOPE, MAX y BAND
2. Si (BAND = VERDADERO)
 - entonces
 - Escribir “Desbordamiento – Pila llena”
 - si no
 - Hacer TOPE $\leftarrow$ TOPE + 1 y PILA[TOPE] $\leftarrow$ DATO
 - {Actualiza TOPE e inserta el nuevo elemento en el TOPE de PILA}
3. {Fin del condicional del paso 2}

Algoritmo 3.4 Quita

Quita (PILA, TOPE, DATO)

{Este algoritmo saca un elemento —DATO— de una estructura tipo pila —PILA—, si ésta no se encuentra vacía. El elemento que se elimina es el que se encuentra en la posición indicada por TOPE}

1. Llamar a Pila_vacia con PILA, TOPE y BAND
2. Si (BAND = VERDADERO)
 - entonces
 - Escribir “Subdesbordamiento – Pila vacía”
 - si no
 - Hacer DATO $\leftarrow$ PILA [TOPE] y TOPE $\leftarrow$ TOPE – 1 {Actualiza TOPE}
3. {Fin del condicional del paso 2}

A continuación se presenta un ejemplo para ilustrar el funcionamiento de las operaciones de inserción y eliminación en pilas.

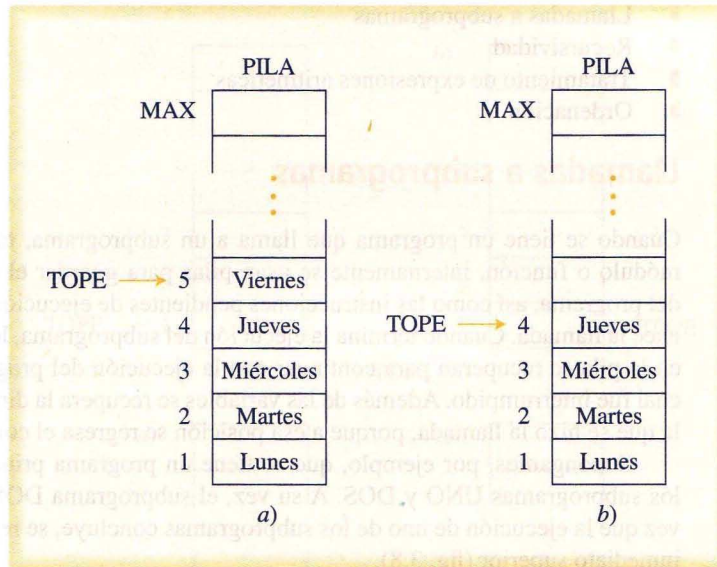
Ejemplo 3.1

Si se insertaran los elementos *lunes*, *martes*, *miércoles*, *jueves* y *viernes* en PILA, la estructura quedaría tal y como se muestra en la figura 3.6a. Ahora bien, si se eliminara el elemento *viernes*, el TOPE apuntaría ahora a *jueves* (fig. 3.6b).

Si en algún momento se quisiera eliminar al elemento *martes*, esto no sería posible ya que sólo se puede tener acceso al elemento que se encuentra en la cima de la pila.

FIGURA 3.6

Interción y eliminación
de pilas.



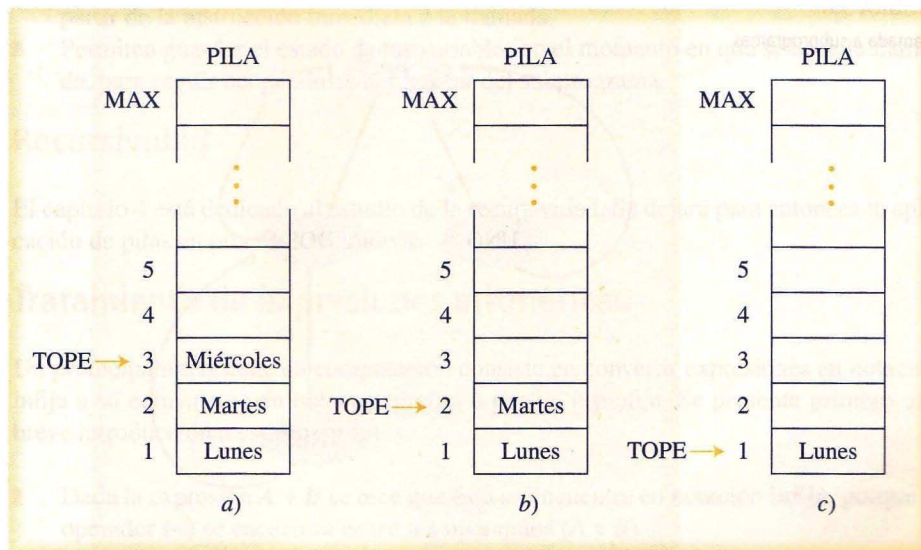
Una forma de resolver este problema es eliminar primeramente los elementos *jueves* y *miércoles*, de esta manera *martes* quedaría ubicado en la cima de PILA y ahora sería posible extraerlo (figuras 3.7a, 3.7b y 3.7c).

3.2.3 Aplicaciones de pilas

Las pilas son una estructura de datos muy usada en la solución de diversos tipos de problemas, en el área de la computación. Ahora se analizarán algunos de los casos más representativos de aplicación de las mismas:

FIGURA 3.7

Interción y eliminación,
aluego de sacar jueves.
aluego de sacar miér-
coles. c) Luego de sacar
martes.



- ▶ Llamadas a subprogramas
- ▶ Recursividad
- ▶ Tratamiento de expresiones aritméticas
- ▶ Ordenación

Llamadas a subprogramas

Cuando se tiene un programa que llama a un subprograma, también conocido como módulo o función, internamente se usan pilas para guardar el estado de las variables del programa, así como las instrucciones pendientes de ejecución en el momento que se hace la llamada. Cuando termina la ejecución del subprograma, los valores almacenados en la pila se recuperan para continuar con la ejecución del programa en el punto en el cual fue interrumpido. Además de las variables se recupera la dirección del programa en la que se hizo la llamada, porque a esa posición se regresa el control del proceso.

Supongamos, por ejemplo, que se tiene un programa principal (*PP*) que llama a los subprogramas *UNO* y *DOS*. A su vez, el subprograma *DOS* llama al *TRES*. Cada vez que la ejecución de uno de los subprogramas concluye, se regresa el control al nivel inmediato superior (fig. 3.8).

Cuando el programa *PP* llama a *UNO*, se guarda en una pila la posición en la que se hizo la llamada (fig. 3.9a). Al terminar *UNO*, el control se regresa a *PP* recuperando previamente la dirección de la pila (fig. 3.9b). Al llamar a *DOS*, nuevamente se guarda la dirección de *PP* en la pila (fig. 3.9c). Cuando *DOS* llama a *TRES*, se pone en la pila la dirección de *DOS* (fig. 3.9d). Después de procesar *TRES*, se recupera la posición de *DOS* para continuar con su ejecución (fig. 3.9e). Al terminar *DOS* se regresa el control a *PP*, obteniendo previamente la dirección guardada en la pila (fig. 3.9f).

Finalmente podemos concluir que las pilas son necesarias en este tipo de aplicaciones por lo siguiente:

FIGURA 3.8

Llamada a subprogramas.

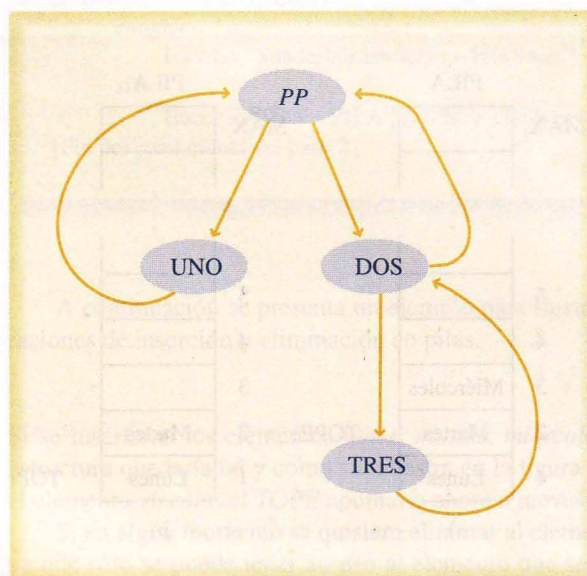
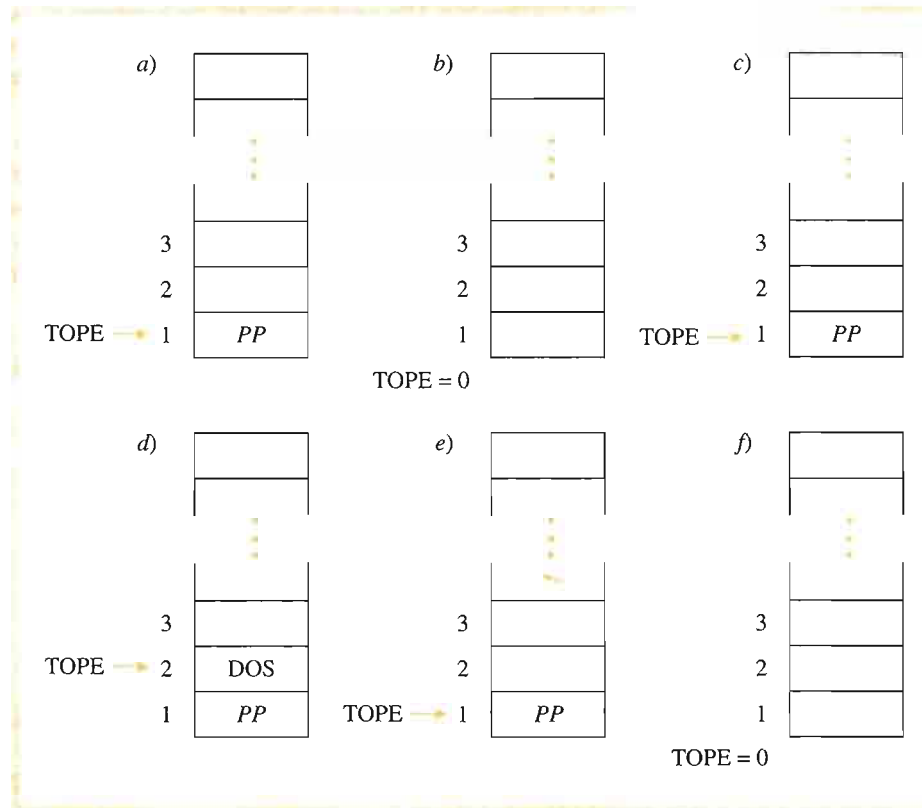


Figura 3.9

Relación de pilas: llamadas a subprogramas.



- Permiten guardar la dirección del programa, o subprograma, desde donde se hizo la llamada a otros subprogramas, para regresar posteriormente y seguir ejecutándolo a partir de la instrucción inmediata a la llamada.
- Permiten guardar el estado de las variables en el momento en que se hace la llamada, para seguir ocupándolas al regresar del subprograma.

Recursividad

El capítulo 4 está dedicado al estudio de la recursividad. Se dejará para entonces la aplicación de pilas en procesos recursivos.

Tratamiento de expresiones aritméticas

Un problema interesante en computación consiste en convertir expresiones en notación infija a su equivalente en notación prefija o posfija o prefija. Se presenta primero una breve introducción a estos conceptos.

- Dada la expresión $A + B$ se dice que ésta se encuentra en notación **infija**, porque el operador (+) se encuentra **entre** los operandos (A y B).

- ▶ Dada la expresión $AB+$ se dice que ésta se encuentra en notación **postfija**, porque el operador (+) se encuentra **después** de los operandos (A y B).
- ▶ Dada la expresión $+AB$ se dice que ésta se encuentra en notación **prefija**, porque el operador (+) está **precediendo** a los operandos (A y B).

La ventaja de usar expresiones en notación postfija o prefija radica en que no son necesarios los paréntesis para indicar orden de operación, ya que éste queda establecido por la ubicación de los operadores con respecto a los operandos.

Para convertir una expresión dada en notación infija a una en notación postfija o prefija se establecen primero ciertas condiciones:

- ▶ Solamente se manejarán los siguientes operadores —se presentan de mayor a menor según sea su prioridad de ejecución—:

Operador	Nombre de la operación
$^$	Potencia
$*$ /	Multiplicación y división
$+$ -	Suma y resta

- ▶ Los operadores de más alta prioridad se ejecutan primero.
- ▶ Si hubiera en una expresión dos o más operadores de igual prioridad, entonces se procesarán de izquierda a derecha.
- ▶ Las subexpresiones que se encuentran entre paréntesis tendrán más prioridad que cualquier operador.

Se presentan a continuación, paso a paso, algunos ejemplos de conversión de expresiones infijas a notación posfija.

Ejemplo 3.2

En este ejemplo se exponen dos casos de traducción de notación infija a posfija. El primero de ellos es una expresión simple, mientras que el segundo presenta mayor grado de complejidad. En la tabla 3.1 se muestran los pasos necesarios para lograr la traducción de la primera expresión, y en la tabla 3.2 los correspondientes a la segunda expresión.

a) Expresión infija: $X + Z * W$

Expresión posfija: $XZW*+$

El primer operador que se procesa durante la traducción de la expresión es la multiplicación, paso 1, debido a que es el de más alta prioridad. Se coloca el operador

TABLA 3.1
Traducción de infija
a posfija

Paso	Expresión
0	$X + Z * W$
1	$X + ZW*$
2	$XZW*+$

tal manera que los operandos afectados por él lo precedan. Para el operador de suma se sigue el mismo criterio, los dos operandos lo preceden. En este caso el primer operando es X y el segundo es ZW^* .

b) Expresión infija: $(X + Z)^* W / T^{\wedge} Y - V$

Expresión postfija: $XZ + W^*TY^{\wedge}/V -$

Tabla 3.2

Traducción de expresión infija a postfija.

Paso	Expresión
0	$(X + Z)^* W / T^{\wedge} Y - V$
1	$XZ + ^*W / T^{\wedge} Y - V$
2	$XZ + ^*W / TY^{\wedge} - V$
3	$XZ + W^* / TY^{\wedge} - V$
4	$XZ + W^* TY^{\wedge} / - V$
5	$XZ + W^* TY^{\wedge} / V -$

En el paso 1 se convierte la subexpresión que se encuentra entre paréntesis por ser la de más alta prioridad. Luego se sigue con el operador de potencia, paso 2, y así con los demás, según su jerarquía. Como consecuencia de que la multiplicación y la división tienen igual prioridad, se procesa primero la multiplicación por encontrarse más a la izquierda en la expresión, paso 3. El operador de la resta es el último que se mueve, paso 5. A continuación se presenta el algoritmo que traduce una expresión infija a otra postfija.

Algoritmo 3.5 Conv_postfija

Conv_postfija (EI, EPOS)

{Este algoritmo traduce una expresión infija —EI— a postfija —EPOS—, haciendo uso de una pila —PILA—. MAX es el número máximo de elementos que puede almacenar la pila}

1. Hacer TOPE $\leftarrow 0$
2. Mientras (EI sea diferente de la cadena vacía) Repetir
 - Tomar el símbolo más a la izquierda de EI. Recortar luego la expresión
 - 2.1 Si (el símbolo es paréntesis izquierdo)
 - entonces {Poner símbolo en PILA. Se asume que hay espacio en PILA}
 - Llamar a Pone con PILA, TOPE, MAX y símbolo
 - si no
 - 2.1.1 Si (el símbolo es paréntesis derecho)
 - entonces
 - 2.1.1.1 Mientras (PILA[TOPE] $\neq$ paréntesis izquierdo) Repetir
 - Llamar a Quita con PILA, TOPE y DATO
 - Hacer EPOS $\leftarrow$ EPOS + DATO

2.1.1.2 {Fin del ciclo del paso 2.1.1.1}
 Llamar a Quita con PILA, TOPE y DATO
 {Se quita el paréntesis izquierdo de PILA y no se agrega a EPOS}
si no
2.1.1.3 Si (el símbolo es un operando)
entonces
 Agregar símbolo a EPOS
si no {Es un operador}
 Llamar a Pila_vacía con PILA, TOPE y BAND
2.1.1.3A Mientras (BAND = FALSO) y (la prioridad del operador sea menor o igual que la prioridad del operador que está en la cima de PILA)
 Repetir
 Llamar a Quita con PILA, TOPE y DATO
 Hacer $EPOS \leftarrow EPOS + DATO$
 Llamar a Pila_vacía con PILA, TOPE y BAND
2.1.1.3B {Fin del ciclo del paso 2.1.1.3A}
 Llamar a Pone con PILA, TOPE, MAX y símbolo
2.1.1.4 {Fin del condicional del paso 2.1.1.3}
2.1.2 {Fin del condicional del paso 2.1.1}
2.2 {Fin del condicional del paso 2.1}
3. {Fin del ciclo del paso 2}
4. Llamar a Pila_vacía con PILA, TOPE y BAND
5. Mientras (BAND = FALSO) Repetir
 Llamar a Quita con PILA, TOPE y DATO
 Hacer $EPOS \leftarrow EPOS + DATO$
 Llamar a Pila_vacía con PILA, TOPE y BAND
6. {Fin del ciclo del paso 5}
7. Escribir EPOS

Cabe señalar que para este algoritmo se maneja la escala de prioridades presentada al inicio de esta sección.

Ejemplo 3.3

En este ejemplo se retoman los casos del ejemplo 3.2 para ilustrar el funcionamiento del algoritmo Conv_posfija.

a) Expresión infija: $X + Z * W$

Expresión posfija: $XZW*+$

En la tabla 3.3 se presentan los pasos necesarios para lograr la traducción deseada siguiendo el algoritmo 3.5.

En los pasos 1, 3 y 5 el símbolo analizado —un operando— se agrega directamente a EPOS. Al analizar el operador +, paso 2, se verifica si en PILA hay operadores con mayor o igual prioridad. En este caso, PILA está vacía; por tanto, se pone el símbolo en el tope de ella. Con el operador *, paso 4, sucede algo similar. En PILA no existen

TABLA 3.3

Traducción de expresión
infija a posfija

Paso	ET	Símbolo analizado	Pila	EPOS
0	$X + Z * W$			
1	$+ Z * W$	X		X
2	$Z * W$	+	+	X
3	$* W$	Z	+	XZ
4	W	*	+ *	XZ
5		W	+ *	XZW
6			+	XZW *
7				XZW * +

operadores de mayor o igual prioridad —la suma tiene menor prioridad que la multiplicación—, por lo que se agrega el operador * a PILA. En los dos últimos pasos, 6 y 7, se extraen de PILA sus elementos, agregándolos a EPOS.

b) Expresión infija: $(X + Z) * W / T \wedge Y - V$

Expresión postfija: $XZ+W*TY^{\wedge}/V-$

En la tabla 3.4 se presentan los pasos necesarios para lograr la traducción deseada, siguiendo el algoritmo 3.5.

Los pasos que se consideran más relevantes son: en el paso 5, al analizar el paréntesis derecho se extraen repetidamente todos los elementos de PILA (en este caso sólo el operador +), agregándolos a EPOS hasta encontrar un paréntesis izquierdo. El paréntesis izquierdo se quita de PILA pero no se incluye en EPOS —recuerde que las expresiones en notación polaca no necesitan de paréntesis para indicar prioridades—. Cuando se trata el operador de división, paso 8, se quita de PILA el operador * y se agrega a EPOS, ya que la multiplicación tiene igual prioridad que la división. Al analizar el operador de resta, paso 12, se extraen de PILA y se incorporan a EPOS todos los operadores de mayor o igual prioridad, en este caso son todos los que están en ella —la potencia y la división—, agregando finalmente el símbolo en PILA. Luego de agregar a EPOS el último operando, y habiendo revisado toda la expresión inicial, se vacía PILA y se incorporan los operadores (en este caso el operador -) a la expresión postfija.

A continuación se presenta el algoritmo para convertir expresiones infijas a expresiones escritas en notación prefija.

Ejemplo 3.4

En este ejemplo se exponen dos casos de traducción de notación infija a prefija. El primero de ellos es una expresión simple, mientras que el segundo presenta mayor grado de complejidad.

a) Expresión infija: $X + Z * W$

Expresión prefija: $+ X * Z W$

TABLA 3.4

Traducción de expresión infija a postfija

Paso	EI	Símbolo analizado	Pila	EPOS
0	$(X + Z) * W / T^{\wedge} Y - V$			
1	$X + Z) * W / T^{\wedge} Y - V$	(	(	
2	$+ Z) * W / T^{\wedge} Y - V$	X	(	X
3	$Z) * W / T^{\wedge} Y - V$	+	(+)	X
4	$) * W / T^{\wedge} Y - V$	Z	(+)	XZ
5	$* W / T^{\wedge} Y - V$	)	(	XZ +
		)		XZ +
6	$W / T^{\wedge} Y - V$	*	*	XZ +
7	$/ T^{\wedge} Y - V$	W	*	XZ + W
8	$T^{\wedge} Y - V$	/	/	XZ + W *
		/	/	XZ + W *
9	$^{\wedge} Y - V$	T	/	XZ + W * T
10	$Y - V$	^	/^	XZ + W * T
11	$- V$	Y	/^	XZ + W * TY
		-	/	XZ + W * TY^
12	V	-	-	XZ + W * TY^ /
		-	-	XZ + W * TY^ /
13		V	-	XZ + W * TY^ / V
14				XZ + W * TY^ / V -

En la tabla 3.5 se presentan los pasos necesarios para lograr la traducción deseada.

Como en el caso de la notación postfija, ejemplo 3.2, aquí también el operador de multiplicación se procesa primero. De la traducción de la expresión, paso 1, resulta el operador precediendo a los operandos. Lo mismo para el operador de suma, paso 2.

b) Expresión infija: $(X + Z) * W / T^{\wedge} Y - V$

Expresión prefija: $-/*+XZW^{\wedge}TYV$

En la tabla 3.6 se presentan los pasos necesarios para lograr la traducción deseada.

Lo primero que se procesa en este caso es la subexpresión que se encuentra entre paréntesis, paso 1. El orden en que se procesan los operadores es el mismo que se siguió

TABLA 3.5

Traducción de expresión infija a prefija

Paso	Expresión
0	$X + Z * W$
1	$X + * ZW$
2	$+ X * ZW$

TABLA 3.6

Traducción de expresión
infija a prefija

Paso	Expresión
0	$(X + Z) * W / T ^ Y - V$
1	$+ XZ * W / T ^ Y - V$
2	$+ XZ * W / ^ TY - V$
3	$* + XZW / ^ TY - V$
4	$/ * + XZW ^ TY - V$
5	$- / * + XZW ^ TYV$

para la conversión de infija a posfija. Por tanto, sería reiterativo volver a explicar paso a paso el contenido de la tabla 3.6. Sin embargo, es de destacar la posición que ocupan los operadores con respecto a los operandos: los primeros preceden a los segundos.

A continuación se incluye el algoritmo de conversión de notación infija a prefija. Este algoritmo se diferencia del anterior básicamente en el hecho de que los elementos de la expresión en notación infija se recorrerán de derecha a izquierda.

Algoritmo 3.6 Conv_prefija

Conv_prefija (EI, EPRE)

{Este algoritmo traduce una expresión en notación infija, EI a prefija, EPRE, haciendo uso de una pila —PILA—}

{TOPE es una variable de tipo entero y MAX representa el máximo número de elementos que puede almacenar la pila}

1. Hacer TOPE $\leftarrow$ 0
2. Mientras (EI sea diferente de la cadena vacía) Repetir
 - Tomar el símbolo más a la derecha de EI recortando luego la expresión
 - 2.1 Si (el símbolo es paréntesis derecho)
 - entonces {Poner símbolo en pila}
 - Llamar a Pone con PILA, TOPE, MAX y símbolo
 - si no
 - 2.1.1 Si (símbolo es paréntesis izquierdo)
 - entonces
 - 2.1.1.1 Mientras (PILA[TOPE] $\neq$ paréntesis derecho) Repetir
 - Llamar a Quita con PILA, TOPE y DATO
 - Hacer EPRE $\leftarrow$ EPRE + DATO
 - 2.1.1.2 {Fin del ciclo del paso 2.1.1.1}
 - {Sacamos el paréntesis derecho de PILA y no se agrega a EPRE}
 - Llamar a Quita con PILA, TOPE y DATO
 - si no
 - 2.1.1.3 Si (símbolo es un operando)
 - entonces
 - Agregar símbolo a EPRE

dad; por tanto, permanecerá en PILA hasta el final del proceso de conversión, paso 15. Cuando se encuentra un paréntesis derecho, paso 9, se agrega directamente a PILA, mientras que cuando el símbolo analizado es un paréntesis izquierdo, paso 13, se extrae repetidamente cada elemento de PILA agregándolo a EPRE, hasta que se encuentra un paréntesis derecho. Éste se retira de PILA y no se agrega a EPRE. Cuando ya se analizaron todos los símbolos de la expresión se procede a quitar de PILA sus elementos, añadiéndolos a EPRE. Finalmente se invierte EPRE para obtener la expresión en notación prefija, paso 17. Para evitar el último paso del algoritmo, invertir la expresión, se podrían ir concatenando los símbolos en EPRE en orden inverso.

Ordenación

Otra aplicación de las pilas se puede ver en el método de ordenación rápida. Como el tema de ordenación es ampliamente tratado en el capítulo 8, se sugiere remitirse a él.

3.2.4 La clase Pila

La **clase Pila** tiene atributos y métodos. Los atributos son la colección de elementos y el TOPE. Los métodos, por otra parte, son todas aquellas operaciones analizadas en la sección anterior —Pila_vací, Pila_llena, Pone y Quita—. En la figura 3.10 se puede observar gráficamente la clase Pila.

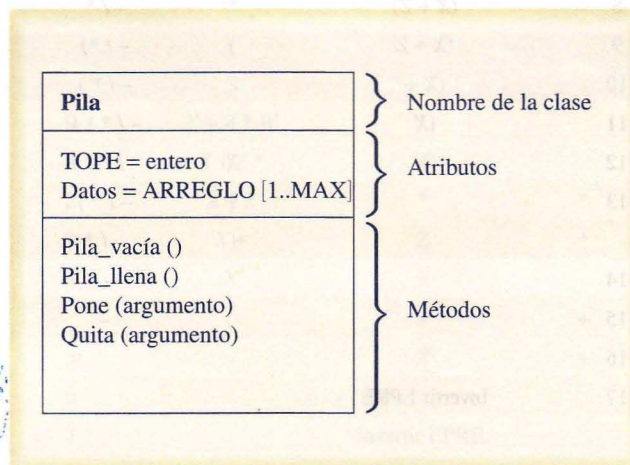
Se tiene acceso a los miembros de un objeto de la clase *Pila* por medio de la notación de puntos. Al asumir que la variable PIOBJ representa un objeto de la clase *Pila* previamente creado, se puede hacer:

PIOBJ.Pila_llena para invocar el método que determina si la pila está llena o no. En este método no hay argumentos, ya que todos los valores requeridos son miembros de la clase.

PIOBJ.Pone(argumento) para insertar un nuevo elemento en la pila. En este método sólo hay un argumento que indica el valor a guardar en la pila; los demás valores requeridos son miembros de la clase.

FIGURA 3.10

Clase Pila.



3.3 COLAS

Una **cola** constituye una estructura lineal de datos en la que los nuevos elementos se introducen por un extremo y los ya existentes se eliminan por el otro. Es importante señalar que los componentes de la cola se eliminan en el mismo orden en el cual se insertaron. Es decir, el primer elemento que se introduce en la estructura será el que se eliminará en primer orden. Debido a esta característica, las colas también reciben el nombre de estructuras **FIFO** (*First-In, First-Out*: el primero en entrar es el primero en salir).

FIGURA 3.11

Ejemplos prácticos
de colas.



Existen numerosos casos de la vida real en los cuales se usa este concepto. Por ejemplo, la cola de los bancos en las que los clientes esperan para ser atendidos —la primera persona de la cola será la primera en recibir el servicio—, la cola de los niños que esperan a veces pacientemente para subir a un juego mecánico, las colas de los vehículos esperando la luz verde del semáforo, las colas para entrar a un cine, teatro o estadio de fútbol, etcétera.

3.3.1 Representación de colas

Las colas, al igual que las pilas, no existen como estructuras de datos estándar en los lenguajes de programación. Este tipo de estructura de datos se puede representar mediante el uso de:

- ▶ Arreglos
- ▶ Listas

Al igual que en el caso de las pilas, en este libro se utilizarán arreglos para mostrar su funcionamiento. Sin embargo, la implementación mediante listas es incluso más sencilla. El lector puede implementar los algoritmos necesarios para colas, después de estudiar el capítulo que se dedica a la estructura lineal de datos.

Cuando se implementan con arreglos unidimensionales, es importante definir un tamaño máximo para la cola y dos variables auxiliares. Una de ellas para que almacene la posición del primer elemento de la cola —FRENTE— y otra para que guarde la posición del último elemento de la cola —FINAL—. En la figura 3.12 se muestra la representación de una cola en la cual se han insertado tres elementos: 111, 222 y 333, en ese orden.

El elemento 111 está en el FRENTE ya que fue el primero que se insertó; mientras que el elemento 333, que fue el último en entrar, está en el FINAL de la cola.

FIGURA 3.12

Representación de colas.

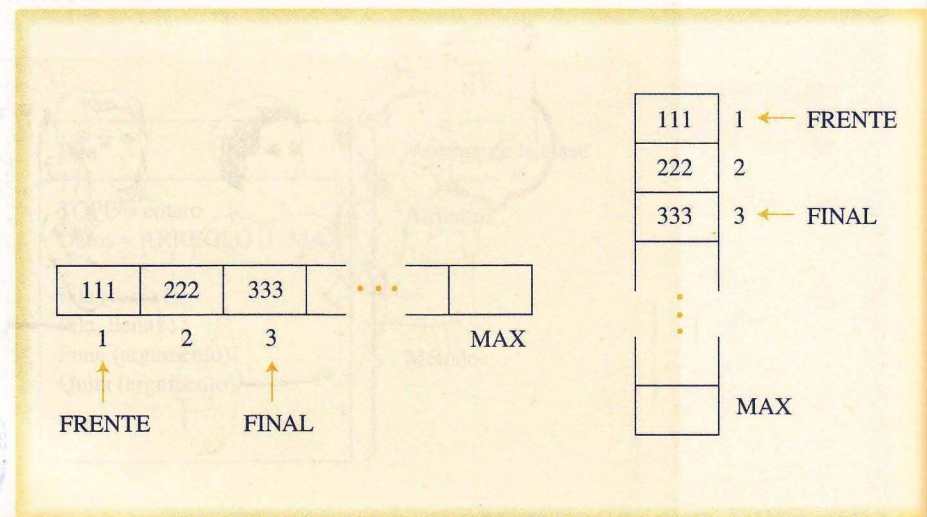
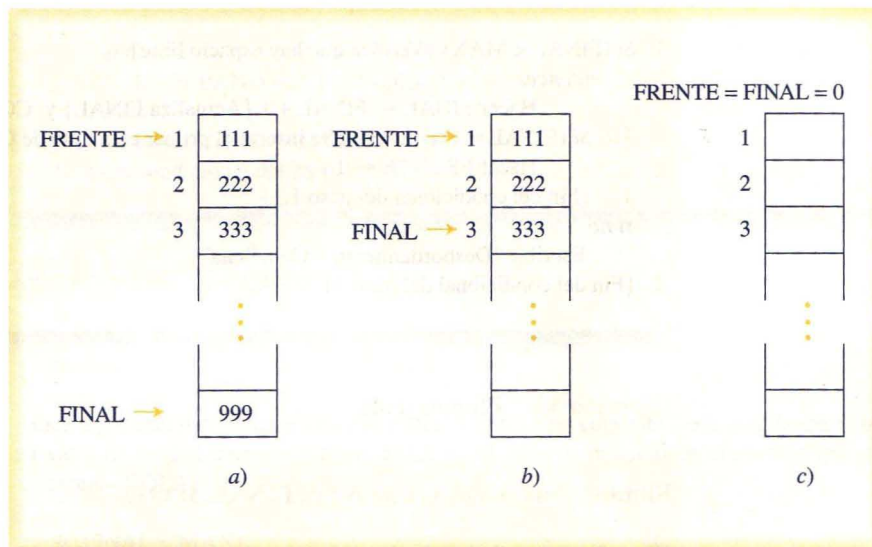


FIGURA 3.13

Representación de colas,
a) Cola llena. b) Cola con
algunos elementos. c) Cola
vacía.



En la figura 3.13, por otra parte, se presentan ejemplos de: a) cola llena, b) cola con algunos elementos y c) cola vacía.

3.3.2 Operaciones con colas

La definición de la estructura de datos tipo cola queda completa al incluir las operaciones que se pueden realizar en ella. Las operaciones básicas que pueden efectuarse son:

- ▶ Insertar un elemento en la cola
- ▶ Eliminar un elemento de la cola

Las inserciones se llevarán a cabo por el FINAL de la cola, mientras que las eliminaciones se harán por el FRENTE —recuerde que el primero en entrar es el primero en salir—.

Considerando que una cola puede almacenar un máximo número de elementos y que además FRENTE indica la posición del primer elemento y FINAL la posición del último, se presentan a continuación los algoritmos correspondientes a las operaciones mencionadas.

Algoritmo 3.7 Inserta_cola

Inserta_cola (COLA, MAX, FRENTE, FINAL, DATO)

{Este algoritmo inserta el elemento DATO al final de una estructura tipo cola. FRENTE y FINAL son los punteros que indican, respectivamente, el inicio y fin de COLA. La primera vez FRENTE y FINAL tienen el valor 0, ya que la cola está vacía. MAX es el máximo número de elementos que puede almacenar la cola}

1. Si $(FINAL < MAX)$ { Verifica que hay espacio libre }
 entonces
 Hacer $FINAL \leftarrow FINAL + 1$ { Actualiza FINAL } y $COLA[FINAL] \leftarrow DATO$
 - 1.1 Si $(FINAL = 1)$ entonces { Se insertó el primer elemento de COLA }
 Hacer $FRENTE \leftarrow 1$
 - 1.2 { Fin del condicional del paso 1.1 }
- si no
 Escribir "Desbordamiento – Cola llena"
2. { Fin del condicional del paso 1 }

Algoritmo 3.8 Elimina cola**Elimina_cola (COLA, FRENTE, FINAL, DATO)**

{ Este algoritmo elimina el primer elemento de una estructura tipo cola y lo almacena en DATO. FRENTE y FINAL son los punteros que indican, respectivamente, el inicio y fin de la cola }

1. Si $(FRENTE \neq 0)$ { Verifica que la cola no esté vacía }
 entonces
 Hacer $DATO \leftarrow COLA [FRENTE]$
 - 1.1 Si $(FRENTE = FINAL)$ { Si hay un solo elemento }
 entonces
 Hacer $FRENTE \leftarrow 0$ y $FINAL \leftarrow 0$ { Indica COLA vacía }
 - si no
 Hacer $FRENTE \leftarrow FRENTE + 1$
 - 1.2 { Fin del condicional del paso 1.1 }
 - si no
 Escribir "Subdesbordamiento – Cola vacía"
2. { Fin del condicional del paso 1 }

Es posible definir algoritmos auxiliares para determinar si una cola está llena o vacía. A partir de estos algoritmos se podrían reescribir los algoritmos 3.7 y 3.8.

A continuación se presentan los algoritmos que permiten verificar el estado de una cola, quedando como tarea sugerida la reescritura de los dos algoritmos anteriores.

Algoritmo 3.9 Cola_vacía**Cola_vacía (COLA, FRENTE, BAND)**

{ Este algoritmo determina si una estructura de datos tipo cola está vacía, asignando a BAND el valor de verdad correspondiente }

1. Si $(FRENTE = 0)$


```

    entonces
        Hacer BAND ← VERDADERO
    si no
        Hacer BAND ← FALSO
2. {Fin del condicional del paso 1}

```

Algoritmo 3.10 Cola_llena

Cola_llena (COLA, FINAL, MAX, BAND)

{Este algoritmo determina si una estructura de datos tipo cola está llena, asignando a BAND el valor de verdad correspondiente. MAX es el número máximo de elementos que puede almacenar COLA}

```

1. Si (FINAL = MAX)
    entonces
        Hacer BAND ← VERDADERO
    si no
        Hacer BAND ← FALSO
2. {Fin del condicional del paso 1}

```

Aquí se incluye un ejemplo para ilustrar el funcionamiento de las operaciones de inserción y eliminación en colas.

Ejemplo 3.6

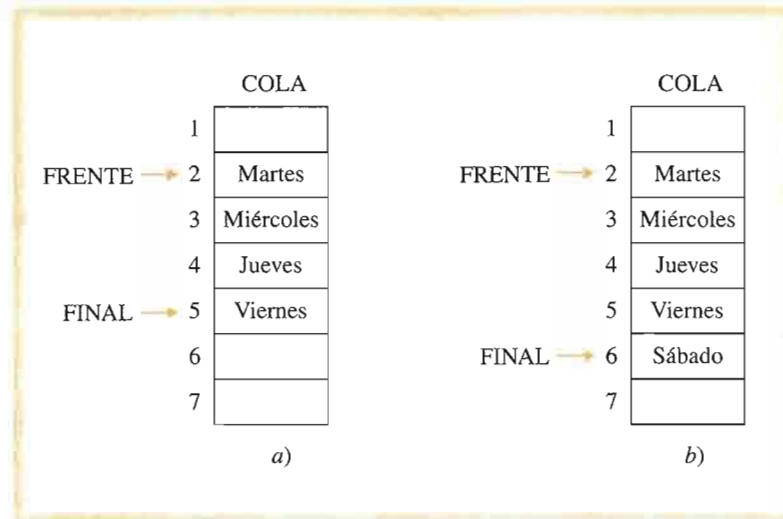
Retome el ejemplo 3.1 de la sección 3.1.2. Se insertan en COLA los elementos: *lunes*, *martes*, *miércoles*, *jueves* y *viernes* —en ese orden—, de modo que la estructura queda como se muestra en la figura 3.14. Para este ejemplo MAX = 7.

FIGURA 3.14
Inserción y eliminación
en COLAS



FIGURA 3.15

Inserción y eliminación en colas. a) Luego de eliminar lunes. b) Luego de insertar sábado.



El elemento *lunes* es el primero que se puede eliminar por ser el primero que insertó en la cola. Luego de la eliminación, FRENTE guarda la posición del siguiente elemento (fig. 3.15a). Si ahora se insertara *sábado*, éste ocuparía ahora la posición siguiente al elemento *viernes* (fig. 3.15b).

Analice lo que ocurre en la cola, si se llevan a cabo las siguientes operaciones:

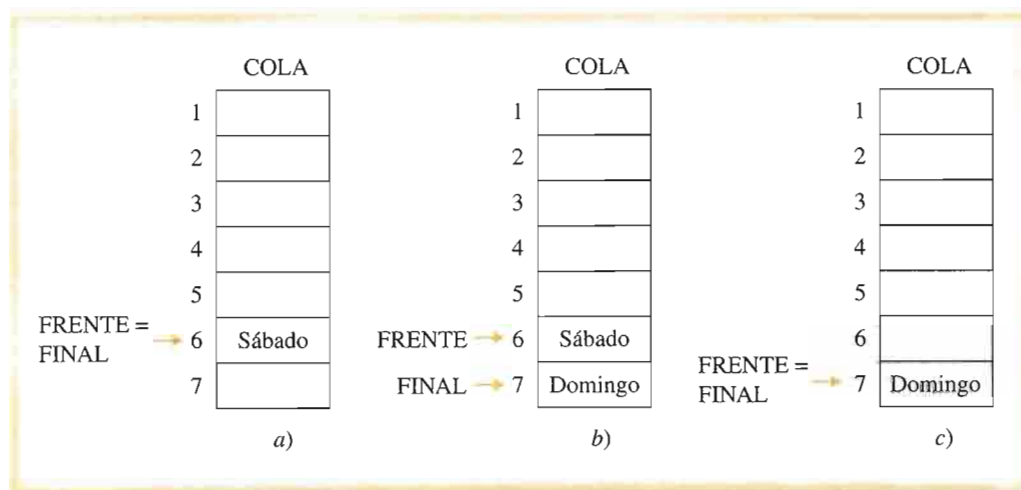
Se eliminan *martes*, *miércoles*, *jueves* y *viernes* (fig. 3.16a).

Se inserta *domingo* (fig. 3.16b).

Se elimina *sábado* (fig. 3.16c).

FIGURA 3.16

Inserción y eliminación en colas. a) Luego de eliminar martes, miércoles, jueves y viernes. b) Luego de insertar domingo. c) Luego de eliminar sábado.



Después de insertar al elemento *domingo*, ya no se pueden insertar nuevos elementos a la cola porque FINAL es igual que MAX (FINAL = MAX = 7). Sin embargo, como lo refleja la figura 3.16c, existe espacio disponible desperdiciado.

Observe que luego de insertar *domingo* se llegó a una situación conflictiva porque a pesar de que hay espacio disponible, no se pueden insertar otros elementos. Este inconveniente se puede superar perfectamente si manejamos las colas en forma circular.

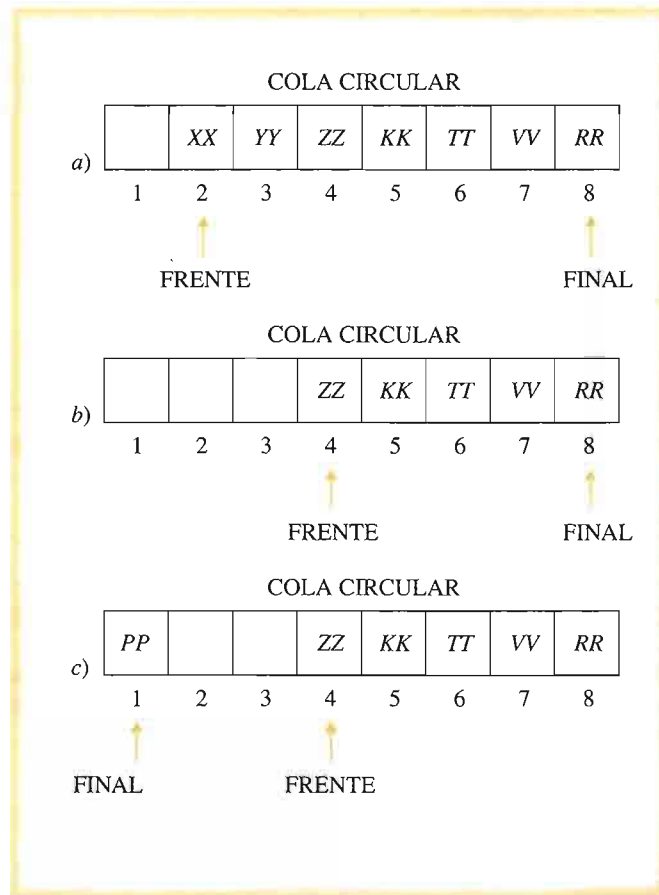
3.3.3 Colas circulares

Una **cola circular** constituye una estructura de datos lineal en la cual el siguiente elemento del último en realidad es el primero. De esta forma se utiliza de manera más eficiente la memoria de la computadora. En la figura 3.17 se muestra la representación gráfica de una cola circular.

En la figura 3.17 se ilustra cómo se actualizan los punteros FRENTE y FINAL en una cola circular, a medida que se insertan o eliminan elementos. En la figura 3.17a la cola tiene algunos elementos (FRENTE = 2 y FINAL = 8). En la figura 3.17b se han eli-

FIGURA 3.17

Representación de colas
a) Frente < Final. b) Frente < Final. c) Frente



minado de la cola dos elementos —primero se quitó *XX* y luego *YY*—, quedando *FRENTE* = 4. Por último, en la figura 3.17c se ha insertado un nuevo elemento —*PP*— en la cola. Como *FINAL* = *MAX* se llevó el apuntador a la primera posición que estaba vacía (*FINAL* = 1). De esta manera se logra mejor aprovechamiento del espacio de memoria ya que al eliminar un elemento la casilla correspondiente de la cola queda disponible para futuras inserciones.

A continuación se presentan los algoritmos de inserción y eliminación en colas circulares.

Algoritmo 3.11 Inserta_circular

Inserta_circular (COLACIR, MAX, FRENTE, FINAL, DATO)

{Este algoritmo inserta el elemento *DATO* al final de una estructura tipo cola circular —*COLACIR*—. *FRENTE* y *FINAL* son los punteros que indican, respectivamente, el inicio y el fin de la cola circular. *MAX* es el número máximo de elementos que puede almacenar *COLACIR*}

1. Si $((FINAL = MAX) \text{ y } (FRENTE = 1)) \text{ o } ((FINAL + 1) = FRENTE)$
 - entonces
 - Escribir “Desbordamiento – Cola llena”
 - si no
 - 1.1 Si $(FINAL = MAX)$
 - entonces
 - Hacer $FINAL \leftarrow 1$
 - si no
 - Hacer $FINAL \leftarrow FINAL + 1$
 - 1.2 {Fin del condicional del paso 1.1}
 - Hacer $COLACIR[FINAL] \leftarrow DATO$
 - 1.3 Si $(FRENTE = 0)$ entonces
 - Hacer $FRENTE \leftarrow 1$
 - 1.4 {Fin del condicional del paso 1.3}
2. {Fin del condicional del paso 1}

Algoritmo 3.12 Elimina_circular

Elimina_circular (COLACIR, MAX, FRENTE, FINAL, DATO)

{Este algoritmo elimina el primer elemento de una estructura tipo cola circular —*COLACIR*— y lo almacena en *DATO*. *FRENTE* y *FINAL* son los punteros que indican, respectivamente, el inicio y fin de la estructura. *MAX* es el tamaño de *COLACIR*}

1. Si $(FRENTE = 0)$ {Verifica si la cola está vacía}
 - entonces
 - Escribir “Subdesbordamiento – Cola vacía”

```

    si no
        Hacer DATO  $\leftarrow$  COLACIR[FRENTE]
    1.1 Si FRENTE = FINAL {Si hay sólo un elemento}
        entonces
            Hacer FRENTE  $\leftarrow$  0 y FINAL  $\leftarrow$  0
        si no
            1.1.1 Si (FRENTE = MAX)
                entonces
                    Hacer FRENTE  $\leftarrow$  1
                si no
                    Hacer FRENTE  $\leftarrow$  FRENTE + 1
            1.1.2 {Fin del condicional del paso 1.1.1}
        1.2 {Fin del condicional del paso 1.1}
    2. {Fin del condicional del paso 1}

```

A continuación se presenta un ejemplo para ilustrar el funcionamiento de las operaciones de inserción y eliminación en colas circulares.

Ejemplo 3.7

En la figura 3.18a se presenta una estructura tipo cola circular de máximo 8 elementos ($MAX = 8$), en la cual ya se han almacenado algunos valores. En la figura 3.18b se muestra el estado de la cola luego de insertar el elemento *NN*.

Si se quisiera insertar otro elemento se presentaría un error de desbordamiento, ya que $FINAL + 1 = FRENTE$.

A continuación se eliminan los valores *XX*, *YY*, *ZZ*, *KK*, *TT* y *VV* en ese orden. La cola queda como se muestra en la figura 3.19.

FIGURA 3.18
Inserción y eliminación en colas circulares. a) Estado inicial de la cola circular. b) Luego de insertar *NN*.

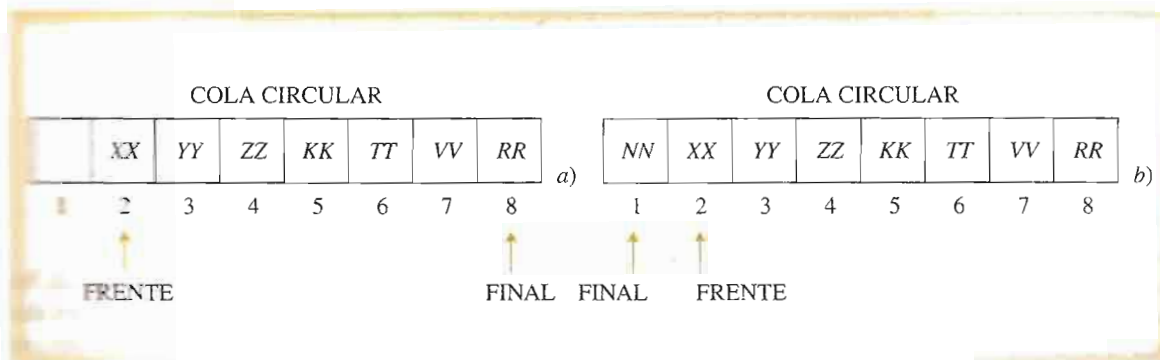
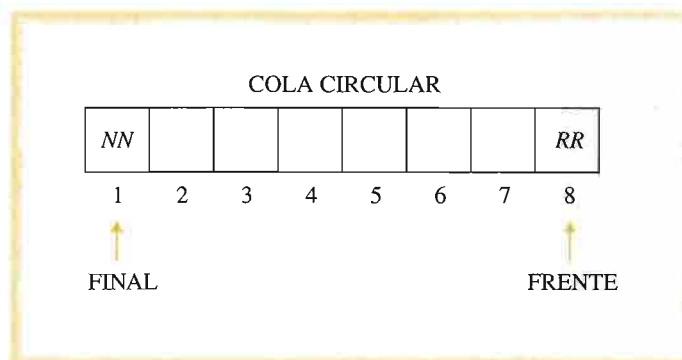


FIGURA 3.19

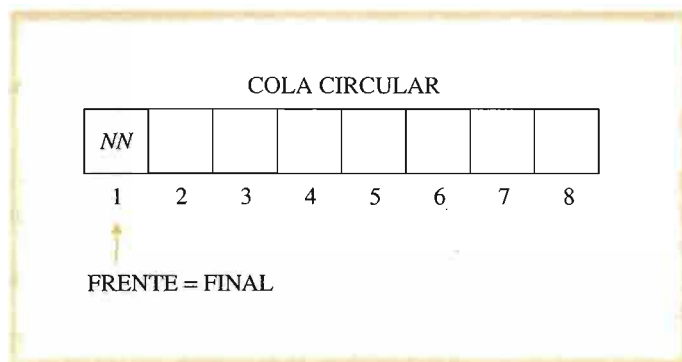
Inserción y eliminación en colas circulares.



Ahora se elimina el siguiente elemento *RR*. Al ser FRENTE = MAX, se le da el valor de 1, figura 3.20.

FIGURA 3.20

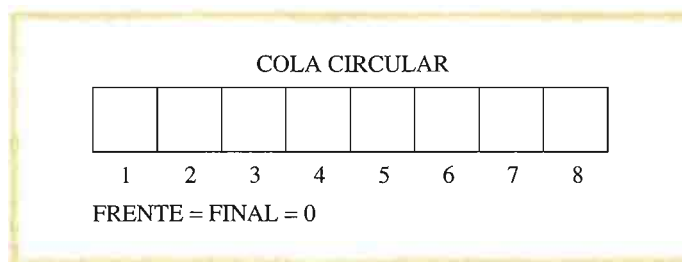
Inserción y eliminación en colas circulares.



Al eliminar *NN*, como FRENTE = FINAL, es decir, sólo quedaba un elemento en la cola, actualizamos los dos punteros en cero. La cola queda vacía, figura 3.21.

FIGURA 3.21

Inserción y eliminación en colas circulares.

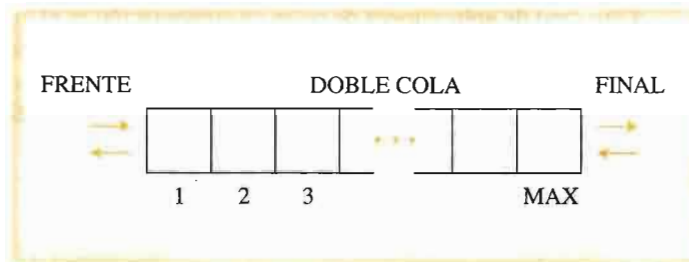


3.3.4 Doble cola

Una **doble cola** o **bicola** es una generalización de una estructura de datos tipo cola. En una doble cola, los elementos se pueden insertar o eliminar por cualquiera de los dos extremos. Es decir, se pueden insertar y eliminar valores tanto por el FRENTE como por el FINAL de la cola. Una doble cola se representa como se muestra en la figura 3.22. Observe que las dos flechas en cada extremo indican que ambas operaciones son posibles.

FIGURA 3.22

Representación de doble cola



Existen dos variantes de las dobles colas:

- ▶ Doble cola con entrada restringida
- ▶ Doble cola con salida restringida

La primera de ellas permite que las eliminaciones se realicen por cualesquiera de los dos extremos, mientras que las inserciones sólo por el FINAL de la cola (fig. 3.23).

La segunda variante permite que las inserciones se realicen por cualquiera de los dos extremos, mientras que las eliminaciones sólo por el FRENTE de la cola (fig. 3.24).

FIGURA 3.23

Doble cola con entrada restringida

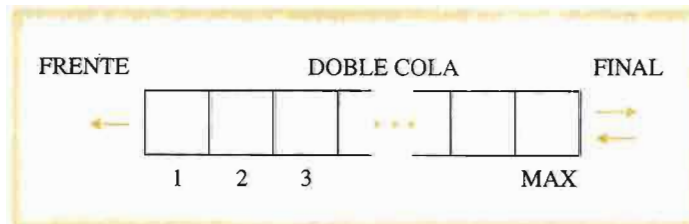
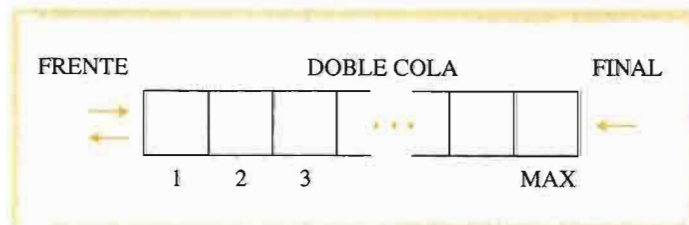


FIGURA 3.24

Doble cola con salida restringida



3.3.5 Aplicaciones de colas

El concepto de cola está ligado a computación. Una aplicación común de las colas se presenta cuando se envía a imprimir algún documento o programa en las colas de impresión. Cuando hay una sola impresora para atender a varios usuarios, suele suceder que algunos de ellos soliciten los servicios de impresión al mismo tiempo o mientras el dispositivo está ocupado. En estos casos se forma una cola con los trabajos que esperan para ser impresos; éstos se procesarán en el orden en el cual fueron introducidos en la cola.

Otro caso de aplicaciones de colas en computación es el que se presenta en los sistemas de tiempo compartido. Varios usuarios comparten ciertos recursos, como CPU y memoria de la computadora. Los recursos se asignan a los procesos que están en cola de espera, suponiendo que todos tienen una misma prioridad, en el orden en el cual fueron introducidos en la cola.

3.3.6 La clase Cola

La **clase Cola** tiene atributos y métodos, como cualquier clase. Los atributos son la colección de elementos y los punteros FRENTE y FINAL. Los métodos, por otra parte, son todas las operaciones analizadas en las secciones anteriores: Cola_vacíá, Cola_llena, InsertaCola y EliminaCola.

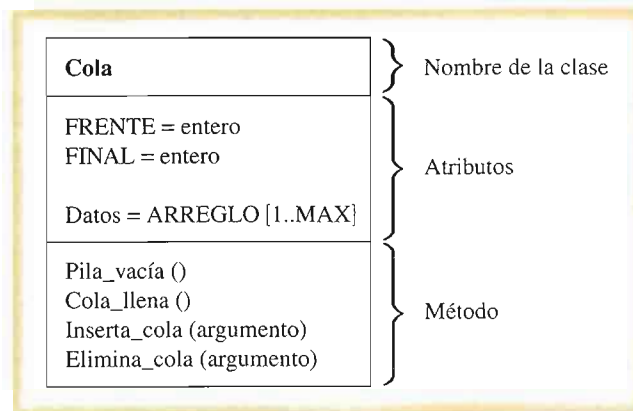
En la figura 3.25 se puede observar gráficamente a la clase Cola. Se tiene acceso a los miembros de un objeto de la clase Cola por medio de la notación de puntos. Cuando se asume que la variable COOBJ es un objeto de la clase Cola, previamente creado, se puede hacer:

COOBJ.Cola_llena para invocar el método que determina si la cola está o no llena. En este método no hay argumentos, ya que todos los valores requeridos son miembros de la clase.

COOBJ.InsertaCola(argumento) para insertar un nuevo elemento en la cola. En este método hay un único argumento, que es el dato a insertar; todos los otros valores requeridos son miembros de la clase.

FIGURA 3.25

Clase Cola.



▼ EJERCICIOS

Pilas

1. Traduzca las siguientes expresiones a notación posfija mediante el algoritmo 3.3.

- a) $X * (Z + W) / (T - V)$
- b) $Z - W * Y + X ^ K$
- c) $X ^ (Z - T) * W$
- d) $(X - Y) * (T - Z)$
- e) $Z / (X + Y * T) ^ W$
- f) $W * (Z / (K - T))$

2. Traduzca las siguientes expresiones a notación prefija con el algoritmo 3.4.

- a) $(X - T) ^ Z$
- b) $(Z * (K - W) + X) ^ Y - T$
- c) $X + T * W / K$
- d) $Z * X - W ^ K$
- e) $(Z - X) ^ Z + (Z - Y) ^ K$
- f) $(W / (X - Z * T) - Y) ^ K$

3. Escriba un programa que lea una expresión en notación infija, y la traduzca a notación posfija.

4. Escriba un programa que lea una expresión en notación posfija, y la traduzca a notación prefija.

5. Escriba un programa que evalúe, con la ayuda de una pila, una expresión aritmética dada en notación prefija.

6. Escriba un programa que evalúe, con la ayuda de una pila, una expresión aritmética dada en notación posfija.

7. Escriba un subprograma que inserte un elemento en una pila. Considere todos los casos que se puedan presentar.

8. Escriba un subprograma que elimine un elemento en una pila. Considere todos los casos que se puedan presentar.

9. Dibuje los distintos estados de una estructura tipo pila si se llevan a cabo las siguientes operaciones. Muestre cómo van quedando la pila y el puntero al tope de la misma. Considere que la pila está inicialmente vacía (TOPE = 0) y tiene una capacidad máxima para 8 elementos

- a) Insertar (PILA, X)
- b) Insertar (PILA, Y)
- c) Eliminar (PILA, Z)
- d) Eliminar (PILA, T)
- e) Eliminar (PILA, U)
- f) Insertar (PILA, V)
- g) Insertar (PILA, W)
- h) Eliminar (PILA, P)
- i) Insertar (PILA, R)

- ¿Con cuántos elementos quedó la pila?
- ¿Hubo algún caso de error (desbordamiento o subdesbordamiento)? Si ocurrió algún error, explíquelo.

10. Escriba un subprograma que elimine los elementos repetidos de una pila. Los elementos repetidos ocupan posiciones sucesivas.
11. Escriba un subprograma que invierta los elementos de una pila.
12. Defina la clase *Pila*, usando algún lenguaje orientado a objetos, con base en los algoritmos presentados para programar los métodos para insertar, eliminar y verificar el estado de la pila.

Colas

13. Sea *C* una cola circular de 6 elementos. Inicialmente la cola está vacía (FRENTE = FINAL = 0). Dibuje el estado de *C* luego de realizar cada una de las siguientes operaciones:
 - a) Insertar los elementos *AA*, *BB* y *CC*
 - b) Eliminar el elemento *AA*
 - c) Insertar los elementos *DD*, *EE* y *FF*
 - d) Insertar el elemento *GG*
 - e) Insertar el elemento *HH*
 - f) Eliminar los elementos *BB* y *CC*
 - ¿Con cuántos elementos quedó *C*?
 - ¿Hubo algún caso de error (desbordamiento o subdesbordamiento)? Si ocurrió algún error, explíquelo.
14. Escriba un subprograma que inserte un elemento en una cola circular. Considere todos los casos que se puedan presentar.
15. Escriba un subprograma que elimine un elemento de una cola circular. Considere todos los casos que se puedan presentar.

16. Utilice una estructura de cola para simular el movimiento de clientes en una cola de espera de un banco —se puede auxiliar con los subprogramas escritos en los ejercicios 14 y 15—.
17. Escriba un subprograma que invierta los elementos de una cola.
18. Defina un algoritmo para insertar un elemento en una doble cola.
19. Defina un algoritmo para eliminar un elemento de una doble cola.
20. Sea C una doble cola circular de 6 elementos. Inicialmente la doble cola está vacía ($\text{FRENTE} = \text{FINAL} = 0$). Dibuje el estado de la cola después de realizar cada una de las siguientes operaciones.
 - a) Insertar por el extremo derecho tres elementos: A , B y C .
 - b) Eliminar por el extremo izquierdo un elemento.
 - c) Insertar por el extremo izquierdo dos elementos: D y E .
 - d) Eliminar por la derecha un elemento.
21. Defina la clase *Cola*, mediante algún lenguaje de programación orientado a objetos, tomando como base para programar los métodos los algoritmos estudiados para insertar, eliminar y verificar el estado de la cola.
22. Se tiene una cola de impresión donde se almacenan las claves de los documentos que se deben imprimir. En la medida en que llega un nuevo trabajo, éste se encola. Cuando la impresora se libera se toma un trabajo de la cola y se imprime. Utilice la clase previamente definida para declarar el objeto *ColaImpresión* y empléelo en el desarrollo de la aplicación descrita.